MedEx: Full-Stack Intern Take-Home Exercise

Clinic Queue & Token System

Thank you for your interest in an internship at MedEx.

MedEx is a healthcare marketplace. Customers in Thailand, Nepal and India use it to book visits to clinics and hospitals. This exercise is a small, self-contained version of a real problem our clinics have. It needs a **backend** and a **frontend**. The logic is the hard part, so get that right first.

How this works

- **Time:** about **8–10 hours** in total. You have **7 days** from the day you receive this. If you run out of time, stop and write in your README what is missing. An honest README counts in your favour.
- **Backend:** Node.js, ideally with TypeScript. Express, Fastify or NestJS are all fine (we use NestJS, so that's a small plus). **Data must survive a server restart**, so use SQLite, a JSON file or any database you like.
- **Frontend:** React, with Vite or Next.js. Any UI library is fine.
- **Data:** `data/clinic.json` (fictional) holds the clinic, its doctors and an example day.
- **Examples:** the data file contains `examples`, each with a list of events and the **exact** result we expect. Write an automated test that replays every example and compares the results.
- **AI tools are allowed.** If you use them, say where in your README. In the interview we will ask you to explain your code and change it live, so make sure you understand every line.

What to send us

1. A link to a Git repository with `backend/` and `frontend/` folders and a commit history we can follow.
2. A `README.md` covering:
    - how to run both parts,
    - how your queue logic works,
    - what you would improve with more time.
3. Tests, at least for the queue logic. They must include the example in the data file.

How we evaluate

Area	What we look for
Logic (most important)	Correct results, including the tricky cases.
API	Clear endpoints, input validation, sensible errors and status codes.
Frontend	Easy to use, and handles loading and error states.
Code quality	Readable, sensibly organised, with tests.
Communication	Your README explains your thinking.

The problem

> *"We have two doctors, patients with appointments, walk-ins and the occasional emergency. Reception wants one fair queue, and patients want to know how long they'll wait."*

Build the queue for **Bodhi Family Clinic** in Kathmandu. Reception checks patients in, each patient gets a token number, doctors press **Call next**, and a screen in the waiting room shows who is being called.

Rules

All times are the clinic's local time, **Asia/Kathmandu (UTC+05:45)**.

1. Check-in and groups. Every patient checks in with a name and age, as one of:

- **Emergency:** always first.
- **Appointment:** has an appointment time. If they check in **more than 15 minutes after** their appointment time, they are treated as a **walk-in**. Exactly 15 minutes late is still on time.
- **Walk-in:** no appointment.

2. Tokens.

- Format: `E-001` for emergency, `A-001` for appointment, `W-001` for walk-in. A late appointment patient gets a `W` token.
- Each letter has its own counter, which starts again at `001` every day.
- "Every day" means the clinic's local day, so it starts at **midnight in Kathmandu**.

3. Call next. When a doctor presses *Call next*, they get the first waiting patient in this order:

1. Emergencies, by check-in time.
2. **Due** appointments, meaning those whose appointment time is no later than now + 10 minutes. Order them by appointment time, then by check-in time.
3. Walk-ins: patients aged **65 or over** first, then everyone else. Within each of those, order by check-in time.
4. If nobody above is waiting, the appointment with the earliest appointment time, even though it isn't due yet.

4. Fairness. Walk-ins must not wait forever. If the last **3** calls, ignoring emergencies and counting both doctors, were all made under rule 3.2 (due appointments), and a walk-in is waiting, call the walk-in instead of the next due appointment.

5. No-shows. A doctor can mark the called patient as a no-show.

- **First time:** the patient goes back to the queue as a **walk-in**. They keep their token, and their check-in time becomes the time of the no-show.
- **Second time:** they leave the queue.

6. Queue and waiting time.

- **Queue order:** the queue shows waiting patients in the order they would be called if the doctors kept pressing *Call next* right now, applying rules 3 and 4.
- **Waiting time:** each patient's estimate is `floor(position / number of doctors) × 12` minutes, with positions counted from 0. So with 2 doctors, positions 0 and 1 show "Next".

7. Doctors.

- A doctor sees one patient at a time.
- They must press *Done* or *No-show* before calling the next one.
- If two doctors press *Call next* at the same moment, they must **never get the same patient**.

Testing with time

Rules depend on the current time, so your backend must let tests set it. For example, accept an `X-Now: 2026-11-02T09:30:00+05:45` header when not running in production, or inject a clock into your services. Explain your approach in your README.

API

```
POST /api/check-ins              { "name", "age", "kind": "appointment" | "walk_in",
                                   "appointmentTime"?, "emergency"? }        -> 201 { "token", ... }
POST /api/doctors/:id/call-next  -> 200 { "patient": {...} } or { "patient": null } if nobody is waiting
POST /api/doctors/:id/done       -> 200
POST /api/doctors/:id/no-show    -> 200
GET  /api/queue                  -> { "nowServing": [{ "doctorId", "token", "name" }],
                                      "waiting": [{ "position", "token", "name", "estimatedWaitMinutes" }] }
```

Validate input:

- `name` is required.
- `age` is between 0 and 120.
- An appointment needs `appointmentTime`.

Return a clear `409` when a doctor action doesn't make sense, for example *Call next* while already seeing a patient.

The example in `data/clinic.json`

It replays a morning. Here is the start:

```
{ "at": "2026-11-02T08:50:00+05:45", "type": "check_in",
  "patient": { "name": "Ram", "age": 45, "kind": "appointment", "appointmentTime": "2026-11-
02T09:00:00+05:45" } }
{ "at": "2026-11-02T08:52:00+05:45", "type": "check_in", "patient": { "name": "Sita", "age": 70, "kind":
"walk_in" } }
{ "at": "2026-11-02T09:00:00+05:45", "type": "call_next", "doctorId": "D1" }
{ "at": "2026-11-02T09:26:00+05:45", "type": "queue" }      // a snapshot of GET /api/queue
```

`expected` has three parts:

- the token each patient got;
- every call in order (`{ "at", "doctorId", "token", "name" }`, where `token: null` means nobody was waiting);
- the waiting list at each `queue` event.

Frontend

Three screens:

1. **Reception:** a check-in form with validation, and the live queue with tokens and waiting times.
2. **Doctor:** choose which doctor you are, see who you're with, and the *Call next*, *Done* and *No-show* buttons.
3. **Waiting-room display:** a large "Now serving" view for a TV.
    - It refreshes by itself without flickering.
    - A newly called token stands out for a few seconds.
    - It must be readable from across a room.

Every screen shows loading and error states. If the server is down, the display says so rather than showing an old queue as if it were current.

Questions?

If something is unclear, make a sensible decision, write it down in your README and carry on. If you're blocked, email us and we'll reply within one working day.

Good luck!